

Bài Ôn tập 2

Mã bài	Đề bài (tiếng Việt)
Bài 1	Yêu cầu: Viết một hàm đếm số ký tự trong chuỗi s và trả về số đó. Nếu s không phải là chuỗi, hãy chuyển nó thành chuỗi rồi mới đếm ký tự.
Bài 2	Yêu cầu: Cho một chuỗi chữ thường, hãy trả về số lần xuất hiện của ký tự 'a'. Nếu đầu vào không phải chuỗi, hãy chuyển sang chuỗi trước.
Bài 3	Yêu cầu: Viết một hàm đếm số từ trong một chuỗi. Một từ là một dãy ký tự không phải khoảng trắng dài nhất có thể. Có thể xuất hiện nhiều dấu cách liên tiếp hoặc dấu cách ở đầu/cuối chuỗi. Nếu đầu vào không phải chuỗi, hãy chuyển sang chuỗi trước.
Bài 4	Yêu cầu: Viết một hàm kiểm tra xem một chuỗi có phải là chuỗi đối xứng (palindrome) hay không, tức là đọc xuôi hay ngược đều giống nhau. Việc kiểm tra là chính xác tuyệt đối (phân biệt chữ hoa/thường và tính cả khoảng trắng). Trả về True hoặc False. Nếu đầu vào không phải chuỗi, hãy chuyển nó thành chuỗi.
Bài 5	Yêu cầu: Cho số nguyên n, hãy trả về tổng các chữ số của nó. Bỏ qua dấu âm (dùng giá trị tuyệt đối).
Bài 6	Yêu cầu: Cho số nguyên không âm n, hãy tạo số lớn nhất và số nhỏ nhất có thể tạo được bằng cách sắp xếp lại các chữ số của n. Trả về dưới dạng danh sách [max_value, min_value] (không dùng tuple). Với số nhỏ nhất, không được để chữ số 0 đứng đầu (ví dụ: 2015 → 5210 và 1025).
Bài 7	Yêu cầu: Nhập số nguyên n, hãy đếm xem trong các chữ số của nó có bao nhiêu chữ số là chữ số nguyên tố (2, 3, 5, 7). Bỏ qua dấu âm.
Bài 8	Yêu cầu: Cho chuỗi thời gian dạng "hh:mm:ss", hãy trả về tổng số giây. Giờ, phút, giây phải là số nguyên với $0 \leq \text{mm} < 60$ và $0 \leq \text{ss} < 60$. Trả về -1 nếu giá trị nằm ngoài phạm vi hợp lệ.
Bài 9	Viết hàm sum_1_to_n(n) trả về tổng của tất cả các số nguyên từ 1 đến n. Nếu n không dương thì trả về 0.
Bài 10	Viết hàm sum_even_1_to_n(n) trả về tổng của tất cả các số chẵn từ 1 đến n (kể cả n). Nếu n không dương thì trả về 0.
Bài 11	Viết hàm sum_mod5_eq2_1_to_n(n) trả về tổng của mọi số nguyên k trong đoạn [1, n] sao cho $k \% 5 == 2$. Nếu n không dương thì trả về 0.
Bài 12	Viết hàm sum_multiples_upto_n(m, n) trả về tổng của tất cả các số từ 1 đến n chia hết cho m. Nếu n không dương hoặc $m = 0$ thì trả về 0.
Bài 13	Tính trung bình cộng của một dãy số.

	Viết hàm <code>average_of_list(a)</code> nhận vào danh sách số <code>a</code> (độ dài ≥ 1) và trả về giá trị trung bình cộng dưới dạng số thực. - Dùng vòng lặp <code>for</code> để tính tổng các phần tử. - Không dùng <code>tuple</code> hoặc <code>dictionary</code>.
Bài 14	Tìm phần tử lớn nhất của một dãy. Viết hàm <code>max_of_list(a)</code> trả về giá trị lớn nhất trong danh sách <code>a</code> (độ dài ≥ 1). - Dùng vòng lặp <code>for</code>. - Không dùng <code>tuple</code> hoặc <code>dictionary</code>.
Bài 15	Tìm phần tử nhỏ nhất của một dãy. Viết hàm <code>min_of_list(a)</code> trả về giá trị nhỏ nhất trong danh sách <code>a</code> (độ dài ≥ 1). - Dùng vòng lặp <code>for</code>. - Không dùng <code>tuple</code> hoặc <code>dictionary</code>.
Bài 16	Đếm số phần tử lớn hơn hoặc bằng <code>x</code>. Viết hàm <code>count_geq(a, x)</code> trả về có bao nhiêu phần tử trong danh sách <code>a</code> lớn hơn hoặc bằng <code>x</code>. - Dùng vòng lặp <code>for</code>. - Không dùng <code>tuple</code> hoặc <code>dictionary</code>.
Bài 17	Kiểm tra dãy có đối xứng (palindrome) hay không. Viết hàm <code>is_palindrome(a)</code> trả về <code>True</code> nếu danh sách <code>a</code> đọc xuôi và đọc ngược giống nhau; ngược lại trả về <code>False</code>. - Dùng vòng lặp <code>for</code> để so sánh các vị trí đối xứng. - Không dùng <code>tuple</code> hoặc <code>dictionary</code>.
Bài 18	Xoay mảng sang phải <code>x</code> bước. Cho danh sách <code>a</code> có độ dài <code>n</code> và số nguyên không âm <code>x</code>, hãy viết hàm <code>rotate_right(a, x)</code> trả về một danh sách mới là kết quả của việc xoay <code>a</code> sang phải <code>x</code> vị trí. Ví dụ, <code>a = [5, 2, 1, 4, 3]</code> và <code>x = 2</code> $\rightarrow$ <code>[4, 3, 5, 2, 1]</code>. - Không thay đổi danh sách đầu vào. - Không dùng <code>tuple</code> hoặc <code>dictionary</code>.
Bài 19	Yêu cầu: Bạn được cho hai danh sách mã sinh viên. Hãy trả về danh sách đã sắp xếp gồm các mã không trùng lặp xuất hiện ở một trong hai danh sách.

	Yêu cầu thêm: Ưu tiên dùng set để loại bỏ trùng lặp. Khai báo hàm: <code>unique_sorted_student_ids(list1, list2) -> list</code>
Bài 20	Bạn được cho hai danh sách: - A: người dùng đã đăng ký - B: người dùng đã thanh toán Hãy trả về 3 danh sách: 1. Người đã thanh toán nhưng chưa đăng ký 2. Người đã đăng ký nhưng chưa thanh toán 3. Người vừa đăng ký và đã thanh toán Mỗi danh sách kết quả cần được sắp xếp theo thứ tự tăng dần.
Bài 21	Cho danh sách số nguyên nums, hãy trả về True nếu có ít nhất một giá trị xuất hiện từ hai lần trở lên trong mảng, và trả về False nếu mọi phần tử đều khác nhau. Khai báo hàm: <code>contains_duplicate(nums) -> bool</code>
Bài 22	Cho mảng số nguyên arr, hãy trả về danh sách bình phương của các số thỏa mãn: - chia hết cho 3, và - không chia hết cho 6. Yêu cầu: - Dùng list comprehension - Không dùng vòng lặp for thông thường Khai báo hàm: <code>squares_div_by_3_not_6(arr) -> list[int]</code>
Bài 23	Chuyển mảng 2 chiều (danh sách các danh sách) thành danh sách 1 chiều theo thứ tự từng hàng, dùng list comprehension. Khai báo hàm: <code>flatten_2d(matrix) -> list</code>
Bài 24	Cho ma trận 2 chiều (danh sách các danh sách) gồm các số nguyên, hãy trả về một danh sách phẳng chỉ chứa các số chẵn, dùng nested list comprehension. Khai báo hàm: <code>even_numbers_in_matrix(matrix) -> list[int]</code>

Bài 25	Viết hàm <code>is_prime(n)</code> trả về True nếu n là số nguyên tố và False nếu ngược lại. Theo quy ước, các số nhỏ hơn 2 không phải là số nguyên tố.
Bài 26	Viết hàm <code>sum_divisors(a)</code> trả về tổng của tất cả các ước dương của a. Nếu $a \leq 0$ thì trả về 0.
Bài 27	Viết hàm <code>is_perfect(n)</code> trả về True nếu n là số hoàn hảo và False nếu không phải. Một số nguyên dương được gọi là hoàn hảo nếu nó bằng tổng các ước dương thực sự của nó (không tính chính nó). Theo định nghĩa, n phải dương; với giá trị không dương thì trả về False.
Bài 28	Yêu cầu: Viết một hàm trả về True nếu chuỗi đầu vào chứa đủ 26 chữ cái tiếng Anh (a–z) ít nhất một lần, không phân biệt chữ hoa/chữ thường; ngược lại trả về False. Bỏ qua các ký tự không phải chữ cái.
Bài 29	Yêu cầu: Cho chuỗi họ tên theo thứ tự Họ, Tên đệm, Tên (ví dụ: "Nguyen Van A"), hãy trả về tên đã sắp lại theo thứ tự Tên Tên đệm Họ (ví dụ: "A Van Nguyen"). Giả sử các thành phần được ngăn cách bởi dấu cách; có thể có nhiều dấu cách liên tiếp. Nếu chỉ có một thành phần thì giữ nguyên.
Bài 30	Yêu cầu: Cài đặt giải mã Atbash cho các chữ cái tiếng Anh. Ánh xạ $a \rightarrow z, b \rightarrow y, \dots, z \rightarrow a$ và tương tự $A \rightarrow Z, \dots, Z \rightarrow A$. Các ký tự không phải chữ cái được giữ nguyên.
Bài 31	Yêu cầu: Một chuỗi được gọi là perfect nếu nó chứa ít nhất một chữ hoa, ít nhất một chữ thường và ít nhất một chữ số. Trả về True nếu chuỗi là perfect; ngược lại trả về False. Nếu đầu vào không phải chuỗi, hãy chuyển thành chuỗi.
Bài 32	Liệt kê tất cả số nguyên tố nhỏ hơn x ($x < 1000$). Viết hàm <code>primes_less_than(x)</code> trả về danh sách tất cả các số nguyên tố nhỏ hơn x. - Dùng các vòng lặp đơn giản và phép kiểm tra số nguyên. - Không dùng tuple hoặc dictionary.
Bài 33	Trong trang trại của Farmer John, Shizuku đếm được n cái chân. Trong trang trại chỉ có: - gà có 2 chân - bò có 4 chân Yêu cầu: Trả về danh sách các tuple (<code>so_ga</code> , <code>so_bo</code>) biểu diễn tất cả các cấu hình có thể. Một cấu hình hợp lệ nếu tổng số chân đúng bằng n.
Bài 34	Bạn được cho mảng a gồm n số nguyên. Trong một bước, bạn có thể chọn $i \neq j$ rồi gán <code>a[i] := a[j]</code> . Bạn có thể thực hiện thao tác này bao nhiêu lần cũng được. Yêu cầu: Trả về True nếu có thể biến đổi mảng để tổng các phần tử là số lẻ. Khai báo hàm: <code>can_make_sum_odd(a) -> bool</code>
Bài 35	Bạn được cho số nguyên dương chẵn n. Hãy xây dựng mảng a có độ dài n sao cho: - $n/2$ phần tử đầu là số chẵn

	- $n/2$ phần tử cuối là số lẻ - tất cả các phần tử đều khác nhau và dương - tổng nửa đầu bằng tổng nửa sau Nếu không thể, trả về (False, []). Nếu có thể, trả về (True, a).
Bài 36	Viết hàm <code>square_matrix_of_n(n)</code> trả về một chuỗi biểu diễn ma trận $n \times n$ mà mọi phần tử đều bằng n. Mỗi hàng cách nhau bởi ký tự xuống dòng, các phần tử trong cùng một hàng cách nhau bởi đúng một dấu cách. Nếu n không dương thì trả về chuỗi rỗng.
Bài 37	Viết hàm <code>rectangle_of_char(n, m, c)</code> trả về một chuỗi biểu diễn lưới gồm m hàng và n cột, trong đó mọi ô đều chứa ký tự c. Các phần tử trong cùng một hàng cách nhau bởi một dấu cách, các hàng cách nhau bởi ký tự xuống dòng. Nếu $n \leq 0$ hoặc $m \leq 0$ thì trả về chuỗi rỗng.
Bài 38	Viết hàm <code>inverted_triangle(n)</code> trả về một tam giác ngược cân bằng gồm các dấu sao *, có độ rộng là n. - n phải là một số nguyên dương lẻ, nếu không thì trả về chuỗi rỗng. - Mỗi dòng có đúng n ký tự, số dấu sao giảm dần 2 đơn vị mỗi dòng cho đến khi còn 1 dấu sao.
Bài 39	Tổng của hai đường chéo trong ma trận $n \times n$. Viết hàm <code>sum_two_diagonals(mat)</code> trả về tổng các phần tử trên đường chéo chính và đường chéo phụ. Nếu n là số lẻ thì không được cộng trùng ô ở giữa. - Dùng vòng lặp. - Không dùng tuple hoặc dictionary.
Bài 40	Tổng các phần tử <code>a[i][j]</code> mà $i + j$ là số chẵn. Viết hàm <code>sum_where_i_plus_j_even(mat)</code> trả về tổng các phần tử ở những vị trí mà $i + j$ là số chẵn (dùng chỉ số bắt đầu từ 0; tính chẵn lẻ cũng tương ứng với cách đánh số từ 1). - Dùng vòng lặp for lồng nhau. - Không dùng tuple hoặc dictionary.
Bài 41	Tổng các phần tử nằm trên hàng lẻ hoặc cột lẻ (đánh số từ 1). Viết hàm <code>sum_odd_rows_or_cols(mat)</code> trả về tổng các phần tử mà chỉ số hàng hoặc chỉ số cột là số lẻ nếu dùng cách đánh số bắt đầu từ 1. (Tức là lấy mọi ô mà hàng lẻ hoặc cột lẻ.) - Dùng vòng lặp. - Không dùng tuple hoặc dictionary.
Bài 42	Trả về cột (chỉ số bắt đầu từ 0) có tổng cột lớn nhất trong ma trận $n \times m$. Viết hàm <code>column_with_max_sum(mat)</code> trả về chỉ số bắt đầu từ 0 của cột có tổng lớn nhất. Nếu có nhiều cột cùng tổng lớn nhất, trả về cột có chỉ số nhỏ nhất. - Dùng vòng lặp để tính tổng từng cột. - Không dùng tuple hoặc dictionary.

Bài 43	Viết hàm <code>zigzag_numbers(n)</code> trả về một chuỗi biểu diễn ma trận $n \times n$ chứa các số từ 1 đến n^2 . Các hàng cách nhau bởi ký tự xuống dòng. Với chỉ số hàng bắt đầu từ 0: hàng chẵn đi từ trái sang phải, hàng lẻ đi từ phải sang trái. Các số trong cùng một hàng cách nhau bởi một dấu cách.
Bài 44	Viết hàm <code>count_solutions_3vars(a, b, c, t)</code> trả về số nghiệm nguyên không âm (x, y, z) của phương trình $a*x + b*y + c*z = t$. Giả sử a, b, c, t là các số nguyên. Nếu $a \leq 0$ hoặc $b \leq 0$ hoặc $c \leq 0$ hoặc $t < 0$ thì trả về 0.
Bài 45	Số lần đổi chỗ kề nhau ít nhất để đưa số 1 duy nhất vào giữa ma trận có kích thước lẻ. Bạn được cho ma trận $n \times n$ (với n là số lẻ). Ma trận chứa đúng một ô có giá trị 1, còn tất cả các ô khác đều là 0. Trong một thao tác, bạn có thể đổi chỗ hai hàng kề nhau hoặc hai cột kề nhau. Hãy tính số thao tác ít nhất để đưa số 1 vào đúng ô trung tâm.
Bài 46	Yêu cầu: Cho danh sách tên sinh viên, hãy trả về 3 tên xuất hiện nhiều nhất. Nếu có ít hơn 3 tên khác nhau, hãy trả về tất cả. Yêu cầu thêm: Ưu tiên dùng <code>map/dictionary</code> . Khai báo hàm: <code>top3_frequent_names(names) -> list[str]</code>
Bài 47	Yêu cầu: Cho danh sách các bộ ba (tuple gồm 3 số nguyên), hãy trả về bộ ba xuất hiện nhiều nhất. Khai báo hàm: <code>most_frequent_triple(triples) -> tuple[int,int,int]</code> - Phát sinh <code>ValueError</code> nếu danh sách đầu vào rỗng.
Bài 48	Cho danh sách số nguyên <code>nums</code> và số nguyên <code>k</code> , hãy trả về <code>True</code> nếu tồn tại hai chỉ số khác nhau <code>i</code> và <code>j</code> sao cho: - <code>nums[i] == nums[j]</code> , và - <code>abs(i - j) <= k</code> Khai báo hàm: <code>contains_nearby_duplicate(nums, k) -> bool</code>
Bài 49	Bạn được cho hai mảng số nguyên 2 chiều <code>items1</code> và <code>items2</code> . Mỗi phần tử có dạng <code>[value, weight]</code> , và <code>value</code> là duy nhất trong từng danh sách. Yêu cầu: Trả về danh sách 2 chiều <code>ret</code> , trong đó mỗi phần tử là <code>[value, total_weight]</code> , với <code>total_weight</code> là tổng trọng lượng của cùng <code>value</code> trong cả hai danh sách. Kết quả cần được sắp theo <code>value</code> tăng dần.